

FINDING ERROR

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 int add(int a, int b);
6 float average(int a, int b, int c);
7 void displayReport();
8
9 extern int totalStudents;
10
11 int main()
12 {
13     int choice = 1
14     // Error: Missing semicolon at the end of the statement.
15     // Solution: int choice = 1;
16
17     int marks = "90";
18     // Error: Assigning a string literal to an int variable.
19     // Solution: int marks = 90;
20
21     int a = 10;
22     int b = 20;
23
24     int result;
25
26     result = add(a);
```

```
27 // Error: Function add() requires two arguments but only one is provided.
28 // Solution: result = add(a, b);
29
30 printf("Sum = %d\n", result);
31
32 if(a = b)
33 // Error: Assignment used instead of comparison.
34 // Solution: if(a == b)
35 {
36     printf("Equal\n");
37 }
38
39 printf("%d\n", score);
40 // Error: Variable 'score' is undeclared.
41 // Solution: Declare score before use or replace with a valid variable.
42
43 int arr[5];
44
45 arr[10] = 100;
46 // Error: Array index 10 is out of bounds for an array of size 5.
47 // Solution: Use indices 0 to 4 only.
48
49 printf("%d\n", arr[-1]);
50 // Error: Negative array index causes undefined behavior.
51 // Solution: Access only valid indices 0 to 4.
52
53 char name[10];
54
```

```
55 strcpy(name, "ProgrammingLanguage");
56 // Error: Buffer overflow. Destination array is too small.
57 // Solution: Increase array size or use strncpy().
58
59 printf("%s\n", name);
60
61 char city[20] = "Jaipur";
62
63 city = "Delhi";
64 // Error: Cannot assign a string literal directly to a character array.
65 // Solution: strcpy(city, "Delhi");
66
67 printf("%s\n", city);
68
69 char user[20];
70
71 scanf("%s", user);
72 // Warning: No width limit specified; may cause buffer overflow.
73 // Solution: scanf("%19s", user);
74
75 if(user == "admin")
76 // Error: Comparing string addresses instead of string contents.
77 // Solution: if(strcmp(user, "admin") == 0)
78 {
79     printf("Valid User\n");
80 }
81
82 int x = 100;
```

```
83  int y = 0;
84
85  printf("%d\n", x/y);
86  // Error: Division by zero.
87  // Solution: if(y != 0) printf("%d\n", x/y);
88
89  int *ptr1 = NULL;
90
91  *ptr1 = 500;
92  // Error: Dereferencing a NULL pointer.
93  // Solution: Allocate memory or assign a valid address before dereferencing.
94
95  int *ptr2;
96
97  printf("%d\n", *ptr2);
98  // Error: Dereferencing an uninitialized pointer.
99  // Solution: Initialize ptr2 before use.
100
101  int *ptr3;
102
103  ptr3 = malloc(sizeof(int));
104
105  *ptr3 = 100;
106
107  free(ptr3);
108
109  printf("%d\n", *ptr3);
110  // Error: Dereferencing a freed pointer (use-after-free).
```

```
111 // Solution: Do not access ptr3 after free(); set ptr3 = NULL.
112
113 free(ptr3);
114 // Error: Double free causes undefined behavior.
115 // Solution: Remove second free() or set ptr3 = NULL after first free().
116
117 int *ptr4;
118
119 ptr4 = malloc(5);
120 // Warning: Allocates only 5 bytes, which may be insufficient.
121 // Solution: ptr4 = malloc(5 * sizeof(int));
122
123 for(int i=0;i<10;i++)
124 {
125     ptr4[i] = i;
126     // Error: Writing beyond allocated memory.
127     // Solution: Allocate sufficient memory and stay within bounds.
128 }
129
130 int *ptr5;
131
132 ptr5 = malloc(sizeof(int));
133
134 ptr5 = malloc(sizeof(int));
135 // Warning: Memory leak. Previous allocation is lost.
136 // Solution: free(ptr5) before reassigning.
137
138 float avg;
```

```
139
140  avg = average(70,80);
141  // Error: Function average() requires three arguments but only two are provided.
142  // Solution: avg = average(70, 80, 90);
143
144  printf("%f\n", avg);
145
146  int age = 17;
147
148  if(age > 18)
149      printf("Not Eligible\n");
150      // Error: Voting eligibility logic is reversed.
151      // Solution: printf("Eligible\n");
152  else
153      printf("Eligible\n");
154      // Error: Voting eligibility logic is reversed.
155      // Solution: printf("Not Eligible\n");
156
157  int num = 13;
158
159  int isPrime = 1;
160
161  for(int i=2;i<num;i++)
162  {
163      if(num%i==0)
164      {
165          isPrime = 1;
166          // Error: Prime flag should be set to 0 when divisor is found.
```

```
167     // Solution: isPrime = 0;
168     break;
169 }
170 }
171
172 if(isPrime)
173     printf("Not Prime\n");
174     // Error: Prime-check output is reversed.
175     // Solution: printf("Prime\n");
176 else
177     printf("Prime\n");
178     // Error: Prime-check output is reversed.
179     // Solution: printf("Not Prime\n");
180
181 displayReport();
182 // Error: Function declared but not defined.
183 // Solution: Provide a definition for displayReport().
184
185 calculateSalary();
186 // Error: Function called without declaration or definition.
187 // Solution: Declare and define calculateSalary().
188
189 printf("%d\n", totalStudents);
190
191 FILE *fp;
192
193 fp = fopen("abc.txt","r");
194
```

```
195  fscanf(fp,"%d",&num);
196  // Error: Possible NULL pointer dereference if fopen() fails.
197  // Solution: Check if(fp != NULL) before using fscanf().
198
199  fclose(fp);
200
201  switch(choice)
202  {
203      case 1
204          // Error: Missing colon after case label.
205          // Solution: case 1:
206          printf("One\n");
207          break;
208
209      case 2:
210          printf("Two\n");
211
212      default:
213          printf("Other\n");
214  }
215
216  char grade = 'AB';
217  // Error: Multi-character constant assigned to char.
218  // Solution: char grade = 'A';
219
220  printf("%c\n", grade);
221
222  int scores[3];
```

```
223
224  scores[0]=10;
225  scores[1]=20;
226  scores[2]=30;
227  scores[3]=40;
228  // Error: Index 3 is out of bounds for an array of size 3.
229  // Solution: Declare scores[4] or remove this assignment.
230
231  int total = 0;
232
233  for(int i=0;i<=3;i++)
234  // Error: Loop accesses scores[3], which is out of bounds.
235  // Solution: for(int i=0;i<3;i++)
236  {
237      total += scores[i];
238  }
239
240  printf("%d\n", total);
241
242  int radius = 5;
243
244  float area;
245
246  area = 2 * 3.14 * radius * radius;
247  // Error: Incorrect formula for area of a circle.
248  // Solution: area = 3.14 * radius * radius;
249
250  printf("%f\n", area);
```

```
251
252 while(choice > 0);
253 // Error: Trailing semicolon creates an empty infinite loop.
254 // Solution: while(choice > 0)
255 {
256     choice--;
257 }
258
259 continue;
260 // Error: continue statement used outside of a loop.
261 // Solution: Remove it or place it inside a loop.
262
263 printf("Program Completed\n");
264
265 return 0;
266 }
267
268 int add(int a, int b)
269 {
270     return a + b;
271 }
272
273 float average(int a, int b, int c)
274 {
275     return (a+b+c)/3;
276 // Warning: Integer division occurs before conversion to float.
277 // Solution: return (a+b+c)/3.0f;
278 }
```

279

280 int totalStudents = 500;